

LECCIÓN 2

CLAUDE CODE PROFESIONAL

Domina la IA agentic: lee tu base de código, ejecuta comandos en la terminal, interactúa con MCP servers, planifica arquitecturas y automatiza flujos completos directamente desde tu IDE.

PREREQUISITOS DEL MÓDULO

¿Qué necesitas antes de comenzar?



Node.js ≥ 18

Verifica con `node --version`. Claude Code es un paquete npm global y requiere Node.js en el PATH del sistema.



VS Code (última versión)

Necesario para la integración con el panel de chat y la extensión "Claude Code" del Marketplace.



Cuenta Anthropic

Claude Pro o API Key activa. Claude Code consume tokens de tu cuenta. Puedes consultar precios en anthropic.com/pricing.



Proyecto de Prueba

Cualquier repositorio local con código. Idealmente con un `package.json` o similar para explorar la instalación en contexto real.

¿QUÉ ES CLAUDE CODE?

Es una herramienta de codificación agentic que opera directamente en la terminal y se integra en tu IDE. A diferencia de un chat web, tiene la capacidad de leer tu base de código de forma autónoma, editar archivos, ejecutar scripts de validación y usar herramientas para completar tareas de ingeniería de software extremo a extremo.

El mismo prompt. Dos resultados completamente distintos.

CONTEXTO SIN GESTIONAR

- Claude ingiere decenas de archivos irrelevantes para la tarea
- Pierde coherencia entre el inicio y el final de tareas largas
- Genera código genérico que no encaja con el stack real del proyecto
- Requiere 3–5 iteraciones adicionales para corregir desviaciones
- Tokens consumidos en ruido: más costo, menos resultado

CONTEXTO BIEN GESTIONADO

- `CLAUDE.md` define el alcance, stack y reglas antes de la primera línea
- `/plan` estructura la ejecución y detecta dependencias antes de actuar
- `@archivo: líneas` focaliza a Claude solo en lo que importa
- `/compact` libera ventana cuando superas el 70% de uso
- Primera iteración coherente con el proyecto — menor costo, mayor precisión



Instalación de Claude Code

0

Prerequisito: Node.js ≥ 18

Verifica con `node --version`. Si no lo tienes, descárgalo desde nodejs.org. Claude Code es un paquete npm y requiere Node.js en el PATH.

1

Instalación CLI

Abre la terminal PowerShell en tu sistema Windows y ejecuta `irm https://claude.ai/install.ps1 | iex`. Verifica la instalación con `claude --version`.

 Guía oficial de instalación

2

Autenticación

Navega a tu proyecto de prueba en la terminal y ejecuta `claude` para iniciar el proceso seguro de OAuth con Anthropic.

3

Integración en VS Code

Instala la extensión oficial "Claude Code" desde el Marketplace de VS Code para habilitar el chat integrado y las sugerencias inline.

SECCIÓN 2

CONFIGURACIÓN AVANZADA DEL IDE

Optimizando VS Code para un flujo ininterrumpido

Extensiones Clave del Ecosistema



Claude for VS Code

Provee el chat integrado y el entendimiento del workspace. Obligatorio para el curso.



GitHub Copilot

Complemento ideal: su autocompletado inline es ultra-rápido mientras Claude maneja tareas de razonamiento profundo.



GitLens

Brinda contexto sobre el historial de git; útil para que la IA entienda la evolución del código a refactorizar.



Error Lens

Resalta errores inline. Al integrarse, Claude puede "ver" los problemas de linter más fácilmente y repararlos en tiempo real.

Configuración Real de Claude Code

Claude Code **no se configura en el settings.json de VS Code**. Sus ajustes viven en archivos propios:

- **Global:** `~/.claude/settings.json` — aplica a todos tus proyectos
- **Por proyecto:** `.claude/settings.json` — overrides del equipo (versionar en git)
- **Local privado:** `.claude/settings.local.json` — overrides personales (no versionar)

```
// ~/.claude/settings.json
{
  "model": "claude-sonnet-4-6",
  "permissions": {
    "allow": ["Bash", "Edit", "Read"],
    "deny": ["Read(.env.secret)"]
  },
  "env": {
    "NODE_ENV": "development"
  }
}
```

Inyección Manual de Contexto en VS Code



Selección Activa

Resalta un bloque de código específico en el editor antes de abrir el chat para que Claude enfoque su análisis únicamente en ese fragmento, ahorrando tokens.



Menciones de Archivos (@)

Usa el símbolo @ en la caja de texto del chat para buscar e incluir proactivamente archivos completos, carpetas o rutas locales en la memoria de la consulta.



Imágenes y Drag & Drop

Arrastra archivos de código o imágenes directamente al panel de chat. Claude puede leer mockups de UI, diagramas arquitectónicos o capturas de errores visuales.



Preparando el Entorno

1

Instalar Extensiones

Navega a la sección de extensiones en VS Code e instala 'Claude Code' y 'Error Lens'.

2

Ajustar settings.json

Abre la paleta de comandos, busca settings.json e inserta las reglas de privacidad y lectura de pestañas abiertas.

3

Validar Interfaz

Abre un archivo de código y usa Ctrl+Shift+P → busca "Claude Code: Open in new tab" para abrir el panel, o haz clic en el ícono de Claude en la barra lateral de VS Code.

SECCIÓN 3

MODOS DE INTERACCIÓN

Chat IDE (VS Code) vs. Terminal CLI

Dos modos de interacción — un modelo conversacional

Claude Code **no tiene ghost text predictivo** (como GitHub Copilot). Su modelo es conversacional: tú describes, él razona y actúa.



Chat VS Code (Extensión)

Contexto: archivo abierto y selección activa en el editor.
Muestra **diffs visuales** lado a lado antes de aceptar cambios.
Soporta **múltiples tabs** de conversación simultáneas.
Razona sobre el fragmento exacto que tú seleccionas.
Ideal para revisión de código y refactors puntuales.



Terminal CLI

Contexto: workspace completo — lee todos los archivos del proyecto.
Acceso al **set completo de comandos**: `/init`, `/loop`, `--worktree`.
Configura **MCP servers** con `claude mcp add`.
Scriptable para **CI/CD, SSH y automatización**.
Ideal para migraciones multi-archivo y tareas autónomas largas.

Chat de VS Code vs Terminal CLI — ¿Cuándo usar cada uno?

CAPACIDAD / ESCENARIO	CHAT VS CODE	TERMINAL CLI
Revisar diffs antes de aceptar cambios	✓ Vista lado a lado integrada	—
Múltiples conversaciones simultáneas	✓ Tabs independientes	— (una sesión por ventana)
Razonar sobre código seleccionado en pantalla	✓ Selección activa como contexto	— (debes referenciar archivos)
Comandos avanzados: <code>/init</code> , <code>/loop</code> , <code>--worktree</code>	—	✓ Set completo disponible
Configurar MCP servers (<code>claude mcp add</code>)	—	✓ Único lugar soportado
Tareas autónomas largas (8+ archivos, migraciones)	Limitado	✓ Sin interrupciones, scriptable



Práctica Híbrida de Generación

Inline

1

Escribe en tu editor '// Función para validar formato de email' y presiona Enter. Deja que autocompleta el código y acéptalo con Tab.

Chat Integrado

2

Selecciona la función, abre el Chat y pide: "Refactoriza usando expresiones regulares sólidas".

Claude CLI

3

Abre la terminal de VS Code, escribe 'claude' y pide: "Genera los tests unitarios para la función que acabo de crear y ejecútalos".

SECCIÓN 4

CONTEXT AWARENESS

Gestión estratégica del contexto para reducir el gasto de tokens y evitar alucinaciones

¿Cómo "lee" Claude tu Proyecto?



Archivo Activo

El archivo abierto en tu editor tiene el mayor peso algorítmico. Es el contexto inmediato que Claude prioriza.



Estructura del Proyecto

Claude mapea package.json, dependencias, carpetas y archivos principales para entender la arquitectura global.



Archivos Mencionados (@)

Claude solo lee lo que tú le indicas explícitamente con @ruta o lo que él decide explorar. Las pestañas abiertas **no** se inyectan automáticamente.



CLAUDE.md

Las reglas y directivas del proyecto definidas en CLAUDE.md guían el comportamiento del agente IA.

Mejores Prácticas del Contexto

HACER (Do's)

- Referenciar solo los archivos necesarios con @.
- Apuntar a rangos de líneas específicos para mayor precisión:
`@src/auth.ts:45-67`
`--file src/middleware.ts:88-105`
- Pedir planes (/plan) antes de generar código masivo.
- Limpiar el historial de chat frecuentemente.

EVITAR (Don'ts)

- Referenciar archivos enteros cuando solo necesitas un fragmento — metes cientos de tokens innecesarios.
- Asumir que el modelo "recuerda" todo infinitamente.
- Resolver múltiples bugs dispares en un solo prompt.
- Llenar el archivo CLAUDE.md con más de 200 líneas.

FLUJO PROFESIONAL AVANZADO

Plan Mode: Investigar Antes de Actuar

1

Activación

Escribe `/plan` en el CLI o usa el botón "Plan" en VS Code. Claude investiga tu codebase sin modificar ningún archivo.

2

Revisión del Plan

El modelo devuelve un documento estructurado con los archivos que modificará, el enfoque propuesto y los riesgos identificados.

3

Aprobación o Ajuste

Revisa el plan. Si estás de acuerdo, responde "procede". Si no, pide ajustes específicos *antes* de que toque una sola línea de código.

4

Ejecución Controlada

Visibilidad total sobre qué cambia y por qué. Reduce en más de un 60% los errores en tareas que afectan múltiples archivos.

El Patrón de Eficiencia: Document & Clear



Mantiene el contexto por debajo del límite sugerido para mantener la precisión cognitiva del modelo.

Cheat Sheet: Comandos Clave

/plan

Antes de cualquier tarea multi-archivo — investiga sin tocar código.

/clear

Al cambiar de tarea o cuando el contexto supera el 60% — borra historial completo.

/compact

Para continuar sin perder el hilo — resume el contexto sin borrar.

/model

Cambiar entre Opus (razonamiento pesado) y Sonnet (velocidad) según la tarea.

/rewind

Deshacer los últimos cambios de archivos hechos por Claude.

/doctor

Diagnosticar problemas de instalación o configuración del entorno.

/usage

Ver tokens gastados y costo estimado de la sesión actual.

Maximizando el Archivo CLAUDE.md

Un archivo de manifiesto ubicado en la raíz que actúa como el 'System Prompt' central del proyecto.

- **Límite:** Mantenerlo por debajo de 200 líneas para no consumir tokens base innecesarios.
- **Instrucciones:** Define comandos de build, convenciones estrictas (ej. "Usa Async/Await, no promesas") y librerías clave.
- **Divulgación Progresiva:** Para repositorios enormes, usa CLAUDE.md para apuntar hacia sub-documentos en otras carpetas.

```
# Reglas del Proyecto
1. Framework: Next.js App Router (App dir)
2. Estilos: Tailwind CSS estricto
3. TypeScript: Forzado, no uses 'any'

# Comandos
- Run: `npm run dev`
- Test: `npm run test`

# Arquitectura
Para reglas de base de datos lee
`docs/DB.md`.
```



Implementando Document & Clear

1

Crear un archivo de control

Pídele a Claude en la CLI: "Analiza la arquitectura actual y guarda los pasos para xxxxxxxx en `plan.md`".

2

Limpiar Memoria

Ejecuta el comando `/clear` en el terminal de Claude. Observa cómo el contexto se libera inmediatamente.

3

Recuperar Contexto Directo

Inicia un nuevo prompt: "Lee `plan.md` y ejecuta únicamente el código para el paso 1".

SECCIÓN 5

RIESGOS, SEGURIDAD Y PRIVACIDAD

Protegiendo el código corporativo de fugas de datos y código alucinado

Privacidad: ¿Qué viaja a la nube?

SÍ se envía

- El código del archivo activo y el fragmento seleccionado.
- Contenido de pestañas abiertas en VS Code.
- El prompt ingresado y logs pegados en el terminal.
- El historial de la sesión CLI en curso.

NO se envía

- Archivos del OS fuera de tu workspace de VS Code.
- Rutas ocultas en el archivo `.claudeignore`.
- Código de repositorios paralelos cerrados.

Controles de Privacidad

Denegación de permisos en settings.json

La clave "deny" en permissions bloquea rutas específicas. Claude no podrá leer esos archivos aunque se lo pidas explícitamente.

```
// .claude/settings.json
{
  "permissions": {
    "allow": ["Read", "Edit", "Bash"],
    "deny": ["Read(.env)", "Read(.env.*)"]
  }
}
```

.claudeignore

Excluye archivos del indexado pasivo. Claude ni siquiera los ve al explorar el workspace — más temprano en la cadena que deny.

Zero Data Retention

En cuentas Team/Enterprise, el código no se retiene ni se usa para entrenar modelos futuros.

 [Configurar privacidad](#)

Alucinaciones Comunes y Controles



Librerías Fantasma

Riesgo: La IA inventa paquetes inexistentes (slopsquatting).

Control: Busca manualmente en npm/PyPI antes de instalar cualquier dependencia sugerida.



APIs Obsoletas

Riesgo: Métodos deprecados de frameworks que ya no existen.

Control: Fija la versión exacta de las librerías en el CLAUDE.md del proyecto.



El Piloto Automático

Riesgo: Aceptar y mergear código sin comprenderlo.

Control: Pull Requests obligatorios con revisión humana peer-to-peer antes de cada merge.



Vulnerabilidades Sutiles

Riesgo: Inyecciones SQL o fuga de datos en código aparentemente "funcional".

Control: Escaneo SAST automatizado, linters de seguridad y suite de tests obligatoria.

Proceso de Validación de Código IA

1

Validar Librerías

Busca en `npm/PyPI` el paquete exacto sugerido antes de instalar. La IA puede inventar nombres plausibles que no existen.

2

Examinar la Lógica

Lee el código línea por línea. Si no entiendes qué hace un bloque, pide a Claude que te lo explique antes de aceptarlo.

3

Ejecutar Tests y Linter

Corre tu suite de pruebas existente y herramientas SAST. El código debe pasar todos los tests antes de mergear.

4

Iterar con la IA

Devuelve los errores al modelo con el mensaje exacto. Claude corrige mejor cuando recibe el stacktrace completo.

5

Responsabilidad Final

Tú firmas el código. El criterio humano de ingeniería es la última línea de defensa, siempre.

SECCIÓN 6

CUSTOM SKILLS Y AUTOMATIZACIÓN

Creando flujos corporativos repetibles y a demanda sin consumir tokens pasivos

¿QUÉ SON LOS AGENT SKILLS?

Un Skill es un flujo de trabajo encapsulado en archivos locales que extiende la terminal de Claude de forma proactiva. Permite transformar un asistente general en un especialista mediante comandos ejecutables (ej. `/auditar-seguridad`), estandarizando el código para todo el equipo de ingeniería.

Ejemplos Prácticos de Skills



Auditor de Endpoints

Comando: `/auditar-endpoints``

Lee controladores, verifica validación de estados HTTP y evita que stacktraces escapen al cliente.



Generador de ADR

Comando: `/crear-adr``

Toma argumentos informales y genera un registro formal de decisión arquitectónica en Markdown.



Creador de Tests Estrictos

Comando: `/generar-tests``

Lee funciones y fuerza a la IA a escribir pruebas usando mocks, adaptado al framework del proyecto.



Optimizador O(N)

Comando: `/optimizar-codigo``

Busca cuellos de botella en bucles, fugas de memoria y devuelve la complejidad algorítmica.

Arquitectura de Carga Eficiente de Skills

Estructura del Directorio

```
.claude/skills/auditar/  
├─ SKILL.md    ← YAML Frontmatter  
├─ CHECKS.md  ← Recursos base  
└─ run-linter.sh ← Scripts
```

Nivel 1 — Ligero

El Frontmatter (YAML) carga pasivamente solo para identificar comandos. Costo: <100 tokens.

Nivel 2 — Bajo Demanda

Las instrucciones Markdown se cargan *solo* cuando el skill es invocado explícitamente. Cero costo pasivo.

Nivel 3 — Externo

Archivos auxiliares y scripts externos se disparan durante la ejecución. Máxima potencia cuando se necesita.

El Frontmatter Explícito (YAML)



name & description

Obligatorios. Definen el nombre del comando CLI y bajo qué palabras clave semánticas la IA reconoce la herramienta.



disable-model-invocation

Si es 'true', la IA NO activará el skill sola. Solo se corre al teclear el comando. Esencial para evitar gastar tokens en segundo plano.



allowed-tools

Limita los permisos de seguridad. Ej: usar "Read, Grep" asegura que la IA pueda auditar un código, pero el sistema le bloquea el permiso de editarlo.



\$ARGUMENTS

Permite pasar parámetros directo desde la consola. Ej: ``/crear-adr $argumentos_sobre_la_base_de_datos``.

Ejemplo: SKILL.md de Auditoría

Este código crea un auditor de endpoints de sólo lectura (Read-only) para asegurar que el código maneja bien los estados HTTP y oculta información sensible.

- Bloqueamos herramientas de edición.
- Obligamos a la llamada manual.

```
---
name: auditar-endpoints
description: Audita manejo
de errores HTTP.
disable-model-invocation:
true
allowed-tools: "Read, Grep,
Glob"
---
# Instrucciones
Actúa como Arquitecto.
Analiza ` $ARGUMENTS ` o el
archivo activo en VS Code.

REGLA: No modifiques
código. Reporta estados
faltantes y posible fuga de
stacktraces.
```



Blindaje y Creación de Skill

1

Privacidad Base

Crea el archivo `.claudeignore` y añade carpetas confidenciales (`.env`, `node_modules/`).

2

Setup del Skill

Crea el directorio `.claude/skills/auditar-endpoints/` y genera un `SKILL.md` dentro.

3

Configurar Frontmatter

Copia el bloque YAML del ejemplo anterior, asegurando `disable-model-invocation: true` para salvar tokens.

4

Ejecución en CLI

Ve a la terminal de VS Code y dispara: `/auditar-endpoints ARCHIVO_A_ANALIZAR` para recibir el análisis.

Resumen de Flujo Diario Óptimo

1

Preparación Segura

Valida `.claudeignore`, cierra
pestañas ruidosas e invoca
`/plan` en el CLI.



2

Generación Dinámica

Combina sugerencias Inline
(código rápido) con CLI
(múltiples archivos).



3

Iteración & VERIFY

Usa herramientas de análisis,
aplica auditorías y recupera
memoria con `/clear`.

"Claude Code es un copiloto avanzado, no un piloto automático ciego. El criterio humano de ingeniería sigue siendo irremplazable para la validación final de la seguridad."

— Principio Fundamental de Programación Asistida

SECCIÓN 7

ECOSISTEMA EXTENDIDO

Conectando herramientas externas con MCP y estandarizando flujos con Plugins

Model Context Protocol (MCP)

Servidores Locales (stdio)

- Procesos que corren directamente en tu máquina de forma segura.
- Ideales para conectarse a herramientas del sistema, scripts propios o bases de datos locales.
- **Ejemplo:** `claude mcp add --transport stdio sqlite -- npx -y @modelcontextprotocol/server-sqlite`

Servidores Remotos (HTTP)

- Servicios que viven en la nube y requieren autenticación (OAuth o API Keys).
- Conectan la IA con tu stack de herramientas empresariales.
- **Ejemplo:** `claude mcp add --transport http github https://mcp.github.com`

Ejemplos de Servidores MCP Destacados



Notion & Slack

Permite a la IA leer wikis, buscar en hilos de comunicación y escribir documentación técnica de los proyectos en tiempo real.



GitHub & Jira

Claude puede buscar tickets, leer issues, cambiar estados y crear Pull Requests directamente sin que salgas de tu terminal.



Bases de Datos

Conexiones a PostgreSQL, Supabase o SQLite para que la IA consulte datos vivos, entienda el esquema o analice registros de logs.



Puppeteer (Navegador)

Automatización web para que la IA realice pruebas End-to-End, tome capturas de pantalla y verifique cambios visuales en la UI.

DEMO PRÁCTICA

Conectando Claude Code con Azure DevOps MCP

1

Genera un Personal Access Token

En Azure DevOps ve a **User Settings** → **Personal Access Tokens**. Crea un token con scope *Work Items (Read & Write)*.

2

Registra el servidor MCP

```
claude mcp add azure-devops -e AZURE_DEVOPS_ORG_URL=https://dev.azure.com/ORG -e AZURE_DEVOPS_PAT=TU-TOKEN  
-- npx @modelcontextprotocol/server-azure-devops
```

3

Verifica la conexión

Ejecuta `/mcp` en Claude Code. Azure DevOps debe aparecer en estado **connected** con las herramientas de Work Items disponibles.

4

Listo para crear historias

Claude puede ahora leer, crear y actualizar Work Items en tu proyecto directamente desde el chat.

ACTIVIDAD

Carga Historias de Usuario en Azure DevOps



Objetivo

Usar Claude Code + Azure DevOps MCP para crear Work Items reales a partir de un `backlog.md` sin abrir el portal de Azure.



Prompt de inicio

"Lee `backlog.md` y crea en Azure DevOps una Historia de Usuario por cada entrada, con su título, descripción y criterios de aceptación. Proyecto: [nombre]."



Entregable

Verifica en el portal de Azure DevOps que los Work Items aparecen creados con los datos correctos y en el sprint o backlog correspondiente.



Bonus: Enriquece

Pide a Claude que agregue etiquetas de prioridad y asigne cada historia al usuario correspondiente según el equipo definido en `specs.md`.

Plugins: Empaquetando el Conocimiento

¿Qué son los Plugins?

Son paquetes distribuibles que agrupan Skills, Hooks, Agentes y servidores MCP en una sola unidad instalable. Actúan como extensiones preconfiguradas de tu entorno y se gestionan fácilmente mediante el comando integrado `/plugin`.

Estandarización

Permiten a los equipos compartir flujos de trabajo sin configuración manual mediante el archivo `plugin.json`.

Espacio de Nombres

Evitan conflictos en comandos usando prefijos. Ej: `/mi-plugin:revisar-codigo`.

Ejemplos Populares

Frontend Design: Genera interfaces visuales atractivas evitando el código genérico.

PR Review Toolkit: Agentes especializados que analizan, testean y critican Pull Requests.

Compound Engineering: Ecosistema de múltiples agentes para investigación profunda y refactorización.

BDG - BUSINESS DEVELOPMENT GROUP

PREGUNTAS Y RESPUESTAS

BDG - BUSINESS DEVELOPMENT GROUP

¡GRACIAS!